Reviewer Pack — Minimal Order of Integer Bases in Boolean Functions and Comm…

Entry eef67c1e02c4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:33:06 UTC

Minimal Order of Integer Bases in Boolean Functions and Communication Complexity Rank Variance

Entry ID: `eef67c1e02c4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 16:33:06 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Integer Bases) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every boolean function f with n variables, the minimal order of an integer basis that generates its dual vector in the canonical form is linearly correlated with the communication complexity rank variance of f's associated matrix representation, such that MinimalOrder(f) = Θ(CommunicationComplexityRankVariance(f)).

Rationale (proposer's reasoning):

> Integer bases are a well-studied concept in number theory, but their application to communication complexity is rare. This conjecture aims to uncover a connection between the algebraic structure of boolean functions and their complexity as measured by communication complexity. If true, it would provide a novel approach for understanding the inherent difficulty of communicating certain boolean functions.

Taxonomy category: `INCONCLUSIVE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c0951f9490735f0e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a boolean function f with n variables, if the correlation coefficient between MinimalOrder(f) and CommunicationComplexityRankVariance(f) is ≥ 0.7 for all seeds, and no seed produces a correlation coefficient < 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order integer bases" AND boolean functions" - "communication complexity rank variance" AND integer basis" - "boolean function dual vector canonical form" AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def matrix_representation(f, n):
        if len(f) != 2**n:
            raise ValueError("Function length must match the number of elements in the matrix repr
        return [[f[i * (1 << (n - j)) + j] for j in range(n)] for i in range(2**(n-1))]

    def dual_vector(matrix):
        n = len(matrix)
        dual = [0] * n
        for i in range(n):
            for j in range(n):
                if matrix[i][j]:
                    dual[j] += 1 << (n - i - 1)
        return dual

    def minimal_order(dual):
        gcd = 0
        for x in dual:
            while x > 0:
                y = gcd % x
                gcd, x = x, y
        return gcd

    def communication_complexity_rank_variance(matrix):
```

```
        n = len(matrix)
        rank = 0
        for i in range(n):
            if any(matrix[j][i] for j in range(n)):
                rank += 1
        return (rank - 1) / (n - 1) if n > 1 else 0

    def generate_random_seeds(num_seeds):
        return [random.randint(2, 10**9) for _ in range(num_seeds)]

    num_instances = 30
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        for _ in range(num_instances):
            f = generate_boolean_function(n)
            matrix = matrix_representation(f, n)
            dual = dual_vector(matrix)
            min_order = minimal_order(dual)
            rank_variance = communication_complexity_rank_variance(matrix)
            results.append((min_order, rank_variance))

    if not results:
        return {
            "metric_name": "minimal_order_vs_rank_variance",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_results"
        }

    min_orders = [r[0] for r in results]
    rank_variances = [r[1] for r in results]
    correlation_coefficient = sum((min_orders[i] - sum(min_orders) / len(min_orders)) * (rank_vari

    return {
        "metric_name": "minimal_order_vs_rank_variance",
        "metric_value": correlation_coefficient,
        "instances_tested": num_instances * len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.7 and all(corr >= 0.5 for corr in [correl
        "counterexample": ""
    }

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        seeds = generate_random_seeds(30)

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
```

```
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(resu
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_too_low\" first_failing
    else:
        print("RESULT: INCONCLUSIVE no_results")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eacd820f.py", line 101, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eacd820f.py", line 64, in run_trial
    matrix = matrix_representation(f, n)
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_eacd820f.py", line 27, in matrix_repres
    return [[f[i * (1 << (n - j)) + j] for j in range(n)] for i in range(2**(n-1))]
             ~^^^^^^^^^^^^^^^^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to provide the required correlation coefficients. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14008
2	preregistration	ollama_remote	glm4:latest	0	0	9114
3	novelty	ollama_remote	glm4:latest	0	0	15231
4	novelty	ollama_remote	glm4:latest	0	0	8941
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14554
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10392
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11151
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11986
9	judge	ollama_remote	glm4:latest	0	0	24425

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 119804 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/eef67c1e02c4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/eef67c1e02c4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/eef67c1e02c4.tar.gz` (if generated)